INTERVIEW PREPARATION — CLOUD & KUBERNETES

Cloud Deployment Deep Dive

Everything about the containerization and cloud infrastructure side of the LLM Router project — what was used, why, how it was actually deployed live to Google Kubernetes Engine, and answers to the 15 technical questions most likely to come up about it.

github.com/msbhullar/llm-router
Maninderjit Singh Bhullar · Built with Claude Code

CHEAT SHEET

Quick Facts

Local & Containerization	
Packaging	Docker (single image, `router_service/`)
Local multi-service	Docker Compose (router + MongoDB)
Local orchestrator	Kubernetes via Minikube
Local Service type	`ClusterIP` (in-cluster only)
Image loading (Minikube)	`minikube image load`, `imagePullPolicy: Never`
Autoscaler	HPA, 1→3 replicas, 50% CPU target

Live Cloud Deployment (GKE)	
Provider / service	Google Cloud — Google Kubernetes Engine (GKE)
Cluster	`llm-router-demo`, zone `us-east1-b`
Node pool	2 × `e2-small` (2 vCPU / 2GB each), resized up from 1
Registry	Artifact Registry (`us-east1-docker.pkg.dev`)
Cloud Service type	`LoadBalancer` (real public IP)
Funding	GCP free-trial credit, budget alert at $0.50/$0.90/$1.00
Lifecycle	Deployed live, evidence captured, torn down — not run 24/7

SECTION 1

What Cloud Tech Was Used, and Why

The deployment side of this project deliberately goes through **four environments**, each adding one real capability the previous one didn't have: a local Python process → Docker Compose → Kubernetes on Minikube → Kubernetes on a real Google Kubernetes Engine (GKE) cluster. Nothing was skipped, and each stage was actually run and verified, not just described.

Technology	Why it was chosen
Docker	Reproducible packaging — the router runs identically regardless of host machine, and it's the prerequisite for every orchestration layer above it. A separate `requirements-router.txt` (no `datasets` library) keeps the container image lean versus the full dev `requirements.txt`.
Docker Compose	One-command local multi-service orchestration (router + MongoDB together) with a shared network — the simplest possible way to prove the two containers talk to each other correctly before adding Kubernetes complexity on top.
Kubernetes	The industry-standard container orchestrator, chosen specifically to demonstrate *self-healing* deployments and *CPU-based autoscaling* — not just "the app runs in a container." Manifests are declarative YAML, which is what makes the same files portable across environments.
Minikube	A single-node local Kubernetes cluster — the cheapest way to prove the manifests are correct before spending any money on real cloud infrastructure. Its main quirk: it has its own container runtime, separate from the host's Docker daemon, so images have to be explicitly loaded into it.
Google Kubernetes Engine (GKE)	A real, managed, multi-node Kubernetes cluster on Google Cloud — chosen over AWS EKS because GKE gives one free zonal cluster (no mandatory control-plane fee) and GCP's free trial credit made a real, live, publicly-reachable deployment possible at zero net cost.
Artifact Registry	Google Cloud's container image registry — required because a cloud cluster's nodes can't see images that only exist on a local machine's Docker daemon (unlike Minikube, which shares a path to the host). The image gets tagged with a registry path and pushed there before GKE can pull it.
LoadBalancer Service	The Kubernetes Service type that actually provisions a real cloud load balancer with a public IP address, in front of the router Deployment — this is specifically what turns "reachable inside the cluster" into "reachable from any browser on the internet."
HorizontalPodAutoscaler (HPA)	Automatically scales the router Deployment between 1 and 3 replicas based on CPU utilization crossing a 50% threshold — demonstrates elastic scaling under load, not just a fixed number of always-on replicas.
ConfigMap / Secret	Kubernetes' two ways of injecting environment configuration into a Pod. Non-secret values (model names, routing threshold) go in a version-controlled ConfigMap; the OpenAI API key goes in a Secret, created by piping it directly from `.env` into `kubectl` so the raw key value is never written into a YAML file or displayed on screen.
GCP Cloud Shell	A free, browser-based, pre-authenticated terminal with `gcloud` and `kubectl` already installed — used to run every cluster-provisioning and deployment command, avoiding any local GCP credential setup.
Budget alerts	A GCP billing budget ("NO Spending") set to notify at $0.50 / $0.90 / $1.00 — a safety net against an unexpected charge, on top of the deliberate decision to tear the cluster down as soon as evidence was captured rather than leaving it running.

> **The throughline:** every one of these was chosen to demonstrate one specific, real capability — not adopted for its own sake. Docker proves packaging; Compose proves multi-service orchestration; Minikube proves the Kubernetes manifests are correct for free; GKE proves those exact same manifests work, unmodified in structure, against real managed infrastructure with a genuine public endpoint.

SECTION 2

How It Was Actually Deployed: The Journey

Step 1 — Prove it locally with Minikube first

Before spending anything on real cloud infrastructure, the full manifest set (`Deployment`, `Service`, `ConfigMap`, `Secret`, `HorizontalPodAutoscaler` for the router; `Deployment`, `Service`, `PersistentVolumeClaim` for MongoDB) was written and verified end-to-end against a local single-node Minikube cluster — self-healing (killing a pod causes Kubernetes to restart it automatically) and autoscaling (the HPA visibly scaling replicas under synthetic CPU load) were both confirmed working here, for free, before touching GCP at all.

Step 2 — Provision a real GKE cluster (and hit real capacity problems)

The first two cluster-creation attempts failed for reasons that had nothing to do with the manifests or the code:

- **Attempt 1** (`us-central1-a`, `e2-medium`) sat in `PROVISIONING` for the better part of 35 minutes before `gcloud container operations list` revealed the real cause: `GCE_STOCKOUT` — Google's own zone simply didn't have enough machine capacity available at that moment. Deleted and retried elsewhere.
- **Attempt 2** (`us-central1-c`, `e2-medium`) stalled in `PROVISIONING` again for 15–20+ minutes with no error message at all — a genuinely different, ambiguous state. After a fixed cutoff, it was abandoned and retried in a third zone rather than waiting indefinitely.
- **Attempt 3** (`us-east1-b`, `e2-small`) provisioned cleanly within a couple of minutes — `STATUS: RUNNING`, kubeconfig auto-generated, `kubectl get nodes` confirmed `Ready`.

This is a real, first-hand example of a class of cloud problem that has nothing to do with application code: **regional capacity is not guaranteed**, and a production system's rollout plan has to account for a single zone being temporarily unavailable.

Step 3 — Push the image, then apply the same manifests

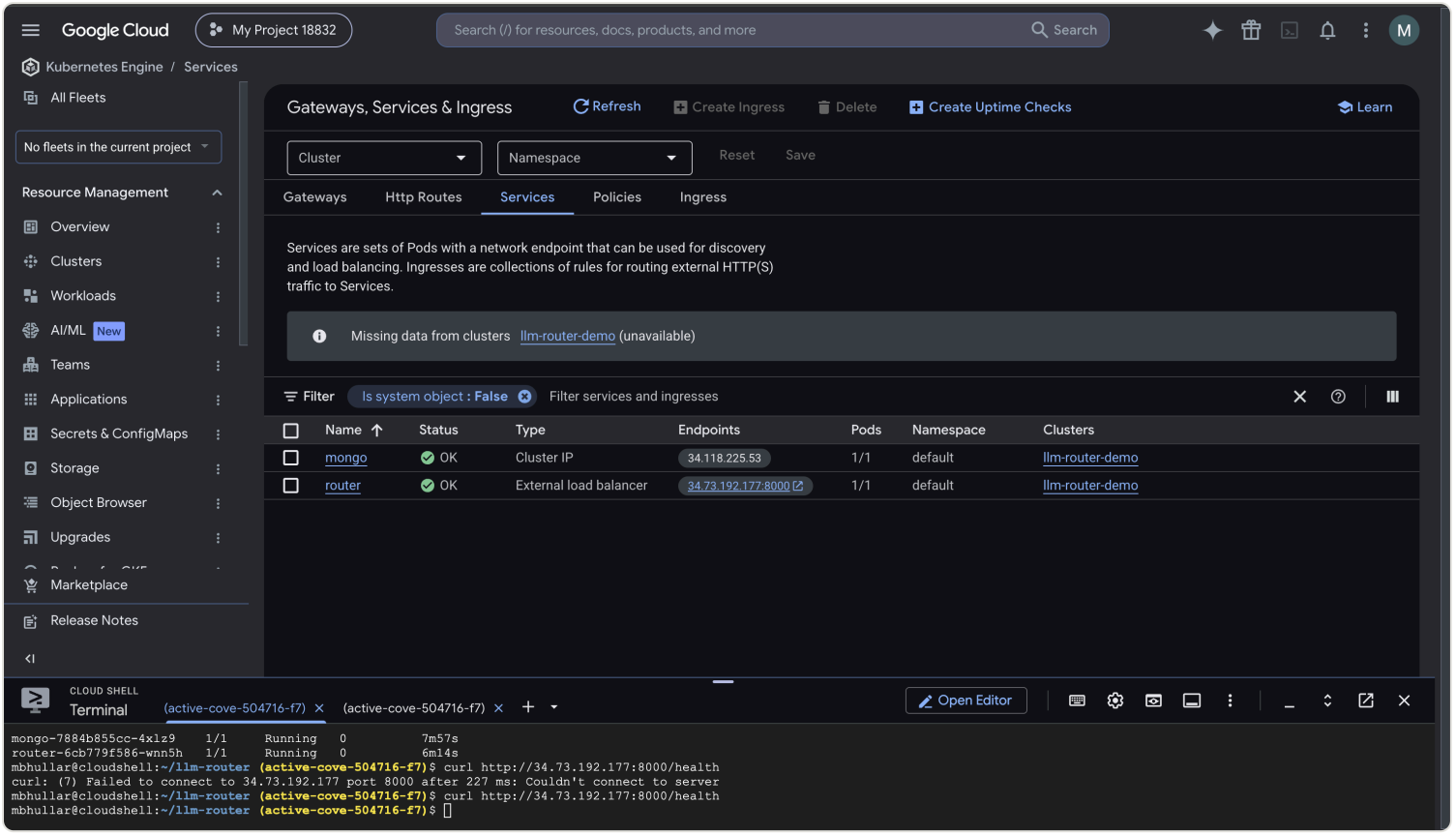
An Artifact Registry repository was created, the local Docker image was tagged with the registry path and pushed, and `k8s/router.yaml` was updated with exactly two cloud-specific changes: the `image:` field now points at the registry (with `imagePullPolicy: Never` removed, since that flag only makes sense for images loaded directly into Minikube's own runtime), and the router's `Service` type changed from `ClusterIP` to `LoadBalancer`. Every other manifest — the Deployment's resource requests/limits, the ConfigMap, the Secret, the HPA, MongoDB's Deployment and PVC — applied completely unchanged.

Step 4 — Diagnose and fix a real pod-scheduling failure

After `kubectl apply -f k8s/`, the router pod sat in `Pending` instead of starting. `kubectl describe pod` showed the actual reason directly in the Events section: `FailedScheduling — 0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory`. The single `e2-small` node (2 vCPU / 2GB RAM total) couldn't simultaneously fit MongoDB's pod and the router's requested 200m CPU / 512Mi memory. The fix was a one-line infrastructure change, not a code change: `gcloud container clusters resize llm-router-demo --zone=us-east1-b --num-nodes=2`. The router pod transitioned `Pending → ContainerCreating → Running` within seconds of the resize completing.

Step 5 — Verify it live, capture evidence, tear it down

With both pods `Running` and the `router` Service showing a real external IP, the deployment was verified from outside GCP entirely — a plain `curl` and a real browser tab hitting the public IP directly, not `localhost` and not `kubectl port-forward`. A hard query submitted through the live demo UI correctly routed to the strong tier, and the live dashboard showed real accumulated cost/latency stats pulled from MongoDB running inside the cluster. The same public endpoint was also confirmed directly inside the GCP Console itself (Kubernetes Engine → Gateways, Services & Ingress), showing the `router` Service as an `External load balancer` with that exact IP — tying the endpoint to the cluster without relying on the terminal at all. Once this evidence was captured, the cluster was deleted (`gcloud container clusters delete`) to stop any further billing against the free-trial credit.



SECTION 3

Every Element & Its Role

Resource	Kind	Role
`router` Deployment	`apps/v1 Deployment`	Runs the FastAPI router container; declares resource `requests` (200m CPU / 512Mi memory) and `limits` (500m CPU / 1Gi memory); pulls config via `envFrom` from the ConfigMap and Secret below.
`router` Service	`v1 Service`	Stable network entry point to the router Pods. `ClusterIP` on Minikube (in-cluster only); `LoadBalancer` on GKE (provisions a real public IP on port 8000).
`router-config`	`v1 ConfigMap`	Non-secret environment variables: `CHEAP_MODEL`, `STRONG_MODEL`, `DIFFICULTY_THRESHOLD`, `MAX_TOKENS`, `MONGODB_URI`. Checked into version control as plain YAML.
`router-secret`	`v1 Secret`	Holds the OpenAI API key. Created imperatively by piping the key straight from `.env` into `kubectl create secret` — never written to a YAML file, never displayed or typed manually.
`router-hpa`	`autoscaling/v2 HorizontalPodAutoscaler`	Watches the router Deployment's CPU utilization; scales replicas between 1 and 3 to keep average CPU utilization near a 50% target. Requires the Deployment's CPU `requests` to exist — without a baseline request, there's nothing for the HPA to compute a percentage against.
`mongo` Deployment	`apps/v1 Deployment`	Runs the stock `mongo:latest` image; mounts a PersistentVolumeClaim so data survives pod restarts.
`mongo` Service	`v1 Service`	`ClusterIP` in both environments — MongoDB is never meant to be reachable from outside the cluster, only from the router Pod via the internal DNS name `mongo`.
`mongo-pvc`	`v1 PersistentVolumeClaim`	1Gi of durable storage for MongoDB's data directory, decoupled from any specific node or Pod's lifecycle.
Artifact Registry repo	GCP resource (not a K8s object)	Cloud-side container registry (`us-east1-docker.pkg.dev/<project>/llm-router-repo`) that GKE's nodes pull the router image from — the cloud equivalent of Minikube's local image cache.
GKE cluster / node pool	GCP resource (not a K8s object)	The actual compute: a managed Kubernetes control plane plus a pool of `e2-small` VM nodes that Pods are scheduled onto. Resized 1→2 nodes mid-deployment to satisfy scheduling requirements.

> **What stayed identical between Minikube and GKE:** the Deployment's resource requests/limits, the ConfigMap, the Secret's structure, the HPA, and everything about MongoDB. **What changed:** exactly two fields in `router.yaml` — the image path/pull policy, and the Service type. That ratio (2 changed fields vs. an entire application's worth of unchanged manifests) is the concrete proof that the "portable by design" claim is real, not just a talking point.

SECTION 4

15 Technical Interview Questions

1 Walk me through how you actually deployed this to the cloud, step by step.

I already had the full Kubernetes manifest set verified locally on Minikube, so the cloud deployment was really about proving those manifests against real infrastructure. I provisioned a GKE cluster using GCP's free trial credit, created an Artifact Registry repository, tagged and pushed my Docker image there, and updated exactly two fields in my router manifest — the image path and the Service type, from `ClusterIP` to `LoadBalancer`. Then I ran `kubectl apply -f k8s/` with everything else unchanged. I hit a real pod-scheduling failure because my single node was too small for both pods, fixed it by resizing the node pool, and once both pods were running I verified the deployment from a completely separate machine — a real browser hitting the public IP, not localhost. Once I had that evidence, I tore the cluster down to stop billing against the free trial.

2 Why Kubernetes at all instead of just running Docker Compose in production?

Docker Compose is great for local development and proving multiple containers can talk to each other, but it doesn't give you self-healing or autoscaling — if a container crashes under Compose, nothing restarts it automatically, and there's no mechanism to add capacity under load. Kubernetes gives you both as declarative, built-in behavior: my Deployment's controller continuously reconciles the actual state against the desired replica count, and my HorizontalPodAutoscaler scales the router between 1 and 3 replicas based on real CPU utilization. I specifically wanted to demonstrate those two capabilities working, not just "the app runs in a container."

3 Explain the difference between ClusterIP and LoadBalancer Service types, and why you used both.

`ClusterIP` is the default Service type — it gives a stable internal DNS name and IP that's only reachable from inside the cluster. I used it for MongoDB in both environments, since the database should never be reachable from outside the cluster, only from the router Pod. `LoadBalancer` goes further: on a cloud provider, it actually provisions a real external load balancer with a public IP address, and forwards traffic from that public IP into the cluster. I used `ClusterIP` for the router on Minikube, since Minikube doesn't have a real cloud load balancer to provision and the Phase 5 goal was only "reachable inside the cluster." On GKE, I switched the router to `LoadBalancer` specifically because that's what actually produces a publicly reachable endpoint — the whole point of the cloud phase.

4 What is a ConfigMap vs. a Secret, and why did you split configuration that way?

Both inject environment variables into a Pod, but they're meant for different sensitivity levels. My ConfigMap holds non-secret values — model names, the routing threshold, the MongoDB connection string — and it's checked into version control as plain YAML, since there's no risk in anyone seeing those values. My Secret holds the OpenAI API key. I created it imperatively, piping the key directly from my `.env` file into `kubectl create secret generic`, so the raw key value was never written into a YAML file, displayed on screen, or typed by hand at any point. Both get pulled into the same Pod via `envFrom`, so the application code doesn't need to know or care which source a given environment variable came from.

5 How did you avoid ever exposing your OpenAI API key in version control or on screen?

The key lives only in my local `.env` file, which is gitignored. To get it into the cluster, I used a shell pipeline that reads directly from that file and pipes it into `kubectl create secret generic router-secret --from-env-file=/dev/stdin` — the value passes through the pipe without ever being echoed to the terminal, written into an intermediate file, or appearing in shell history as a literal argument. The Secret object itself is base64-encoded at rest in the cluster, and I never committed a Secret manifest with a real value to Git — only the ConfigMap, which by design has nothing sensitive in it.

6 Walk me through the pod scheduling failure you hit on GKE. What caused it and how did you diagnose and fix it?

After applying my manifests, the router pod stayed in `Pending` instead of starting. I ran `kubectl describe pod` and looked at the Events section, which gave the exact reason: `FailedScheduling — 0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory`. My cluster had a single `e2-small` node — 2 vCPU and 2GB of RAM total — and between MongoDB's pod and the router's requested 200m CPU / 512Mi memory, there wasn't enough headroom left on that one node for the scheduler to place the router. The fix wasn't a code or manifest change at all; it was an infrastructure change — I resized the node pool from 1 to 2 nodes with `gcloud container clusters resize`. Within seconds of the resize completing, the router pod moved from `Pending` to `ContainerCreating` to `Running`. It's a clean example of Kubernetes' scheduler doing exactly what it's supposed to do — refusing to place a Pod where its declared resource requests can't actually be satisfied, rather than silently overcommitting a node.

7 What is a HorizontalPodAutoscaler, and what does "1→3 replicas at 50% CPU" actually mean mechanically?

An HPA is a Kubernetes controller that watches a metric — in my case, CPU utilization — on a target Deployment, and adjusts the replica count to keep that metric near a target value. Concretely, mine is configured with `minReplicas: 1`, `maxReplicas: 3`, and a target average CPU utilization of 50%. "50% CPU utilization" is measured against each Pod's requested CPU — 200m in my case — not against the node's total capacity. So if the router Pods' actual average CPU usage climbs past 100m (50% of the 200m request), the HPA calculates how many replicas would bring utilization back down near that target and scales up, up to the max of 3. It scales back down the same way once load drops. This is also exactly why the Deployment needs an explicit CPU `request` set — without one, there's no baseline for the HPA to compute a percentage against, and it simply can't function.

8 What's the difference between resource requests and limits in Kubernetes, and why do both matter?

A `request` is what the scheduler guarantees is reserved for the container when deciding which node to place it on — it's the number that caused my pod-scheduling failure when there wasn't 200m CPU / 512Mi memory free on the node. A `limit` is the hard ceiling the container is allowed to consume at runtime — if it tries to use more memory than its limit, it gets OOM-killed; if it tries to use more CPU, it gets throttled, not killed. My router Deployment requests 200m CPU / 512Mi memory and limits at 500m CPU / 1Gi memory. Requests matter for scheduling and are what the HPA measures against; limits matter for protecting the node from a single misbehaving container starving its neighbors. Setting only one or neither is a common real-world misconfiguration — without requests, the HPA can't work and the scheduler can overcommit nodes; without limits, one runaway Pod can degrade everything else on its node.

9 Why did you hit GCE_STOCKOUT errors, and what does that actually mean?

My first cluster-creation attempt in `us-central1-a` sat in `PROVISIONING` for about 35 minutes before `gcloud container operations list` revealed the real status: `GCE_STOCKOUT`. That means Google's own infrastructure in that specific zone didn't have enough physical machine capacity available at that moment to satisfy my node pool's requested VM type — it has nothing to do with my account, my quota, or my configuration. It's a good reminder that "the cloud" isn't infinite, undifferentiated capacity; specific zones can genuinely run tight on specific machine types at specific times. My fix was pragmatic: delete the stuck cluster and retry in a different zone, which is also the standard real-world mitigation — multi-zone or multi-region deployment plans exist partly because any single zone can become temporarily capacity-constrained.

10 What changed in your manifests between Minikube and GKE, and why couldn't you just reuse the exact same files unmodified?

Exactly two fields in one file, `router.yaml`. First, the image reference: Minikube has its own container runtime that can't see the host machine's Docker images unless you explicitly run `minikube image load`, so locally I referenced the image by its local tag with `imagePullPolicy: Never`. GKE's nodes have no access to my laptop at all, so the image has to live somewhere they can pull from — I pushed it to Artifact Registry and pointed the manifest at that registry path, removing `imagePullPolicy: Never` since it only makes sense for a locally-loaded image. Second, the Service type, from `ClusterIP` to `LoadBalancer`, to actually get a public IP. Every other manifest — the Deployment's resource requests and limits, the ConfigMap, the Secret's structure, the HPA, and everything about MongoDB — applied completely unchanged. That's the concrete proof that Kubernetes manifests really are portable by design, not just portable in theory.

11 What is Artifact Registry, and why do you need a container registry at all for a cloud cluster?

Artifact Registry is Google Cloud's managed container image registry — the cloud equivalent of Docker Hub, scoped to a GCP project. A cloud cluster's nodes are separate physical machines with no access to any developer's local Docker daemon, so unlike Minikube — which I can explicitly load a locally-built image into via `minikube image load` — GKE's nodes need to pull the image from somewhere they can actually reach over the network. I created a repository, authenticated Docker to it with `gcloud auth configure-docker`, tagged my image with the registry's path, and pushed it there before GKE could deploy it. Without this step, my Deployment would sit in `ImagePullBackOff` forever — the nodes would have no way to find the image at all.

12 What is imagePullPolicy, and why was Never correct for Minikube but wrong for GKE?

`imagePullPolicy` controls when Kubernetes tries to pull a container image versus using one already present on the node. `Never` means "don't attempt to pull from any registry at all — the image must already exist locally on the node," which only makes sense when you've explicitly loaded the image yourself, as I did with `minikube image load`. On GKE, the nodes never had my image loaded onto them any other way — the only path for them to get it is pulling it from Artifact Registry, so `Never` would have made every pod fail immediately with `ErrImageNeverPull`. Removing that field lets the default policy apply, which pulls from the registry path specified in the image field — exactly what a real cloud deployment needs.

13 How does Kubernetes achieve "self-healing," concretely, in your Deployment?

A Deployment doesn't just create Pods once — it continuously runs a reconciliation loop comparing the desired state (my declared replica count) against the actual state (what's really running), and takes action whenever they diverge. If a Pod crashes, gets OOM-killed for exceeding its memory limit, or a node it's running on goes down, the Deployment's controller notices the actual replica count has dropped below desired and schedules a replacement automatically — no human intervention, no alert needing a manual restart command. I didn't have to write any of that logic myself; it's the core behavior Kubernetes provides just by using a Deployment instead of a bare Pod, which is exactly why I used one for both the router and MongoDB instead of running raw containers.

14 Why did you tear the cluster down instead of leaving it running, and how do you manage cloud cost/billing risk?

Both GKE and AWS EKS incur ongoing costs the moment a cluster exists — GKE's control plane is free for one zonal cluster, but the underlying VM nodes are billed continuously whether or not there's real traffic. This project has no real users, so leaving infrastructure running 24/7 just to keep a public URL alive isn't a good tradeoff, especially as a student already paying for other subscriptions. My plan going in was deliberately "deploy for real, prove it, capture evidence, tear down" rather than either skipping a real deployment entirely or leaving one running indefinitely. On the risk side, I set a GCP billing budget alert at $0.50/$0.90/$1.00 as a safety net on top of that plan, and used a small, cheap node type (`e2-small`) rather than over-provisioning. If a specific situation calls for a live public URL again — like before a particular interview — I can redeploy the exact same way in well under an hour.

15 **If this had real production traffic, what would you change about the cloud setup?**

Several things. I'd put an Ingress with a managed TLS certificate in front of the router instead of a bare `LoadBalancer` on plain HTTP, so traffic is encrypted and I can route multiple services/domains through one entry point. I'd move MongoDB off a self-managed in-cluster Deployment onto a managed service like MongoDB Atlas, since running your own database in Kubernetes means you own backups, failover, and patching yourself — a managed service is the right call once there's real data to protect. I'd deploy across multiple zones (or use GKE's regional cluster mode) so a single zone's capacity issue — like the `GCE_STOCKOUT` I actually hit — can't take the whole system down. And I'd wire up a real CI/CD pipeline to build, push, and roll out new images automatically on merge, instead of the manual `docker build` / `push` / `kubectl apply` sequence I ran by hand for this deployment.